

Summary

Memoization (not memorization) is an optimization technique that can speed up recursive algorithms that have overlapping subproblems by remembering the previous results of identical calculations. Memoization is a common technique in the field of dynamic programming. By trading computer memory usage for improved runtime, memoization makes some otherwise intractable recursive functions possible.

However, memoization won't prevent stack overflow errors. Keep in mind that memoization is not a replacement for using a simple iterative solution. Code that uses recursion for the sake of recursion is not automatically more elegant than non-recursive code.

Memoized functions must be pure—that is, they must be deterministic (returning the same values given the same arguments each time) and not have side effects (affecting anything about the computer or program outside of the function). Pure functions are often used in functional programming, which is a programming paradigm that makes heavy use of recursion.

Memoization is implemented by creating a data structure called a *cache* for each function to memoize. You can write this code yourself, but Python has a built-in `@functools.lru_cache()` decorator that can automatically memoize the function it decorates.